

ROTEIRO DE LABORATÓRIO 4

Da comanda ao módulo testado: laços, funções e pytest

Aula 4, 1 de setembro · duração prevista: 2h30

O que você vai construir

Este roteiro pega o arquivo da Prática 5 da aula passada, que funcionava mas tinha um defeito silencioso, e o transforma em um módulo com funções separadas e uma suíte de testes automatizados.

O ponto alto não é o código novo, é o momento em que um teste falha de propósito e aponta exatamente o defeito que a turma tinha encontrado olhando a tela. A partir daí, esse defeito não volta mais sem alguém ser avisado.

AO FINAL DESTES ROTEIRO VOCÊ TERÁ

Um arquivo comanda.py com quatro funções, nenhuma delas imprimindo nada.

Um arquivo test_comanda.py com cinco testes cobrindo as regras do seu domínio.

O registro da saída do pytest com um teste falhando, e depois com todos passando.

A pasta aula04 enviada ao repositório da equipe.

Antes de começar

Nada novo além de uma biblioteca, que é instalada no passo 1. O restante já está na sua máquina desde a Aula 1.

- Python 3.12 ou superior.
- Visual Studio Code com a extensão Python.
- O repositório da equipe clonado, com a pasta aula03 da semana passada.
- O arquivo pratica5_comanda.py da Aula 3, que serve de ponto de partida.

Passo a passo

Os passos 1 a 3 são aquecimento com laços. Os passos 4 a 6 constroem o módulo. Os passos 7 e 8 são a parte que importa: o teste que falha e a correção. O passo 9 fecha no Git.

1

Preparar a pasta e instalar o pytest 10 min

NO TERMINAL

```
cd ads-2026-2-barbearia
mkdir aula04
cd aula04
```

```
# ambiente isolado so para este projeto
python -m venv .venv
```

```
# ativar: no macOS e Linux
source .venv/bin/activate
```

```
# ativar: no Windows
# .venv\Scripts\activate
```

```
pip install pytest
```

SAÍDA ESPERADA

Successfully installed pytest-8.x.x

POR QUE ISSO IMPORTA

O ambiente virtual é uma pasta com uma cópia isolada do Python e das bibliotecas. Ele existe para que a versão do pytest deste projeto não interfira em outro projeto seu.

Quando o ambiente está ativo, o nome dele aparece entre parênteses no começo da linha do terminal. Se você fechar o terminal, precisa ativar de novo.

ATENÇÃO

A pasta .venv NAO vai para o Git. Ela tem milhares de arquivos e é recriada em qualquer máquina com um comando. O arquivo .gitignore da equipe já cuida disso.

CONFIRA ANTES DE SEGUIR

O terminal mostra o nome do ambiente entre parênteses e o pytest foi instalado sem erro.

2

Aquecimento: os laços 20 min

Antes de mexer na comanda, cinco exercícios curtos de laço. Crie o arquivo e resolva um item por vez, rodando depois de cada um.

PRATICA1_LACOS.PY

```
servicos = ["corte", "barba", "sobancelha", "hidratacao"]
precos = {"corte": 35, "barba": 25, "sobancelha": 15, "hidratacao": 40}
comanda = ["corte", "barba", "corte", "pintura"]
```

```
# 0. percorra servicos com for e imprima cada um

# 1. imprima a mesma lista numerada a partir de 1, com enumerate

# 2. some com um for os precos dos itens da comanda
#     que existem no dicionario precos

# 3. usando while, descubra quantos cortes de 35 cabem em 200

# 4. percorra a comanda e imprima so os itens do catalogo,
#     usando continue para pular os outros
```

NO TERMINAL

```
python pratica1_lacos.py
```

POR QUE ISSO IMPORTA

No item 1, enumerate(servicos, start=1) devolve a posição e o item de uma vez, e evita você criar um contador na mão.

No item 3, o while precisa de uma linha que aproxime o fim. Se você esquecer de subtrair do saldo, o programa trava e não avisa. Para interromper, tecle Ctrl mais C.

CONFIRA ANTES DE SEGUIR

Os cinco itens rodam e imprimem alguma coisa. O item 2 deve somar 95, porque pintura não está no catálogo.

3

Ver a armadilha do remove com os próprios olhos 10 min

Este trecho parece correto e não dá erro nenhum. Rode antes de ler a explicação.

```
COLE NO FIM DO PRATICA1_LACOS.PY
itens = ["corte", "pintura", "tintura", "barba"]

for item in itens:
    if item not in ("corte", "barba"):
        itens.remove(item)

print(itens)
```

SAÍDA ESPERADA

```
['corte', 'tintura', 'barba']
```

ATENÇÃO

Sobrou tintura, e nenhuma mensagem apareceu. Ao remover pintura da posicao 1, tintura desceu para a posicao 1, mas o laço já tinha avançado para a posicao 2. O item foi pulado. Guarde a regra: nunca altere a colecao que esta sendo percorrida.

A CORRECAO: MONTE UMA LISTA NOVA

```
itens = ["corte", "pintura", "tintura", "barba"]

limpos = []
for item in itens:
    if item in ("corte", "barba"):
        limpos.append(item)

print(limpos)
```

SAÍDA ESPERADA

```
['corte', 'barba']
```

CONFIRA ANTES DE SEGUIR

Você viu a saída errada aparecer sem nenhum aviso, e a saída certa depois da correção.

4

Criar o módulo com a primeira função 20 min

Agora começa o trabalho de verdade. Crie comanda.py na pasta aula04 e traga o catálogo do domínio da SUA equipe. Se a equipe não é de barbearia, troque os nomes e os valores.

COMANDA.PY

```
from decimal import Decimal

PRECOS = {
    "corte": Decimal("35.00"),
    "barba": Decimal("25.00"),
    "sobrancelha": Decimal("15.00"),
    "hidratacao": Decimal("40.00"),
}

def itens_validos(comanda, precos=PRECOS):
    """Devolve apenas os itens que existem no catalogo."""
    validos = []
    for item in comanda:
        if item in precos:
            validos.append(item)
    return validos
```

ATENÇÃO

Repare no que NAO tem aqui: nenhum print. Uma funcao que imprime nao pode ser testada, porque o teste nao consegue conferir o que foi para a tela. Quem imprime e quem chama.

POR QUE ISSO IMPORTA

O texto entre tres aspas logo abaixo do def e a docstring. Ela aparece quando alguem passa o mouse sobre o nome da funcao no VS Code, e serve de documentacao.

PRECOS em maiusculas e a convencao de Python para constante: um valor que o programa nao deve alterar.

CONFIRA ANTES DE SEGUIR

O arquivo comanda.py existe, tem o catálogo da sua equipe e a função itens_validos, sem nenhum print.

5

Completar as outras três funções 25 min

Cada função faz uma coisa só e devolve um valor. Repare que subtotal chama itens_validos, e que fechar chama as duas anteriores.

ACRESCENTE AO COMANDA.PY

```
def subtotal(comanda, precos=PRECOS):
    """Soma o preco dos itens validos."""
    total = Decimal("0.00")
    for item in itens_validos(comanda, precos):
        total += precos[item]
    return total

def desconto(comanda, valor, minimo=3):
    """10 por cento a partir de minimo itens VALIDOS."""
    if len(itens_validos(comanda)) >= minimo:
        return valor * Decimal("0.10")
    return Decimal("0.00")

def fechar(comanda):
    """Devolve subtotal, desconto e total."""
    sub = subtotal(comanda)
```

```
desc = desconto(comanda, sub)
return {"subtotal": sub, "desconto": desc, "total": sub - desc}
```

ATENÇÃO

A linha do desconto e a correção do defeito da Aula 3. Na semana passada a condição usava `len(comanda)`, que contava também o item que não foi cobrado. Agora ela usa `len(itens_validos(comanda))`.

Se a regra chata da sua equipe for outra, é ela que entra aqui. Uma equipe de biblioteca teria multa por dia de atraso, uma de estoque teria alerta de estoque mínimo.

POR QUE ISSO IMPORTA

O parâmetro `minimo=3` tem valor padrão, então quem chama pode omitir. Valor padrão só pode ser algo que não muda: número, texto, tupla ou `None`. Lista e dicionário, nunca, porque são criados uma única vez e passam a ser compartilhados entre todas as chamadas.

CONFIRA ANTES DE SEGUIR

As quatro funções estão escritas. Nenhuma delas tem `print`. O arquivo ainda não imprime nada quando você roda.

6

Rodar o módulo direto e entender a linha do `dunder main`

15 min

Como nenhuma função imprime, rodar o arquivo agora não mostra nada. Acrescente o bloco abaixo no fim, e repare que ele fica fora de qualquer função.

NO FIM DO COMANDA.PY

```
if __name__ == "__main__":
    conta = fechar(["corte", "barba", "corte", "pintura"])
    for chave, valor in conta.items():
        print(f"{chave:<9} R$ {valor:.2f}")
```

NO TERMINAL

```
python comanda.py
```

SAÍDA ESPERADA

```
subtotal  R$ 95.00
desconto  R$ 9.50
total     R$ 85.50
```

POR QUE ISSO IMPORTA

Essa linha separa rodar o arquivo de importar o arquivo. Quando você executa `python comanda.py`, a variável `__name__` vale `"__main__"` e o bloco roda. Quando outro arquivo escreve `from comanda import fechar`, a variável vale `"comanda"` e o bloco não roda.

Sem ela, o `pytest` dispararia esse `print` toda vez que importasse o seu módulo para testar. É exatamente o que acontece no próximo passo.

CONFIRA ANTES DE SEGUIR

O comando `python comanda.py` imprime as três linhas do fechamento, com os valores do seu domínio.

7

O primeiro teste 20 min

Crie `test_comanda.py` na mesma pasta. O nome precisa começar com `test_`, é assim que o `pytest` encontra o arquivo.

```
TEST_COMANDA.PY
from decimal import Decimal

from comanda import fechar, itens_validos, subtotal

def test_subtotal_soma_apenas_o_que_esta_no_catalogo():
    assert subtotal(["corte", "barba", "pintura"]) == Decimal("60.00")
```

```
NO TERMINAL
python -m pytest -q
```

SAÍDA ESPERADA

```
. [100%]
1 passed in 0.01s
```

POR QUE ISSO IMPORTA

Um teste tem três passos: preparar a situação, executar a função e verificar o resultado. Aqui os três cabem em uma linha, mas a ordem é sempre essa.

O nome do teste é documentação. `test_subtotal_soma_apenas_o_que_esta_no_catalogo` diz a regra sem que ninguém precise ler o corpo da função.

ATENÇÃO

Se aparecer `ModuleNotFoundError: No module named comanda`, você está rodando o `pytest` de fora da pasta `aula04`. Entre nela com `cd` antes de rodar.

CONFIRA ANTES DE SEGUIR

O `pytest` respondeu 1 passed. Se respondeu erro de importação, confira em que pasta você está.

8

Fazer o teste falhar de propósito 20 min

Este é o passo mais importante do roteiro. Um teste que você nunca viu falhar não prova nada: pode estar passando por acidente. Vamos reintroduzir o defeito da Aula 3 e observar o teste reagir.

Primeiro, escreva o teste que cobre o defeito

```
ACRESCENTE AO TEST_COMANDA.PY
def test_item_fora_do_catalogo_nao_conta_para_o_desconto():
    conta = fechar(["corte", "barba", "pintura"])
    assert conta["desconto"] == Decimal("0.00")
```

Agora quebre o `comanda.py`, trocando uma linha

```
NO COMANDA.PY, DENTRO DE DESCONTO
# comente a linha correta:
# if len(itens_validos(comanda)) >= minimo:

# e coloque a linha errada da Aula 3:
if len(comanda) >= minimo:
```

```
NO TERMINAL
python -m pytest -q
```

SAÍDA ESPERADA

```
.F [100%]

> assert conta["desconto"] == Decimal("0.00")
E AssertionError: assert Decimal('6.0000') == Decimal('0.00')

1 failed, 1 passed in 0.02s
```

ATENÇÃO

Leia a mensagem com calma. A comanda tem tres itens, mas so dois foram cobrados. O desconto de 10 por cento incidiu sobre 60 reais e deu 6 reais. E exatamente o defeito que a turma encontrou olhando a tela na aula passada.

A diferenca e que agora ele foi apontado por uma maquina, em dois centesimos de segundo, sem ninguem precisar reparar.

Desfaça a quebra e confirme o verde

```
VOLTE A LINHA CORRETA
if len(itens_validos(comanda)) >= minimo:
```

```
NO TERMINAL
python -m pytest -q
```

SAÍDA ESPERADA

```
.. [100%]
2 passed in 0.01s
```

CONFIRA ANTES DE SEGUIR

Você viu a saída vermelha com o AssertionError e depois a verde. Anote as duas no documento da equipe.

9

Completar a suíte 25 min

Faltam três testes. O último é o que mais pega gente na vida real: a coleção vazia, que ninguém lembra de conferir na mão.

```
ACRESCENTE AO TEST_COMANDA.PY
def test_desconto_de_dez_por_cento_a_partir_de_tres_itens():
    conta = fechar(["corte", "barba", "corte"])
    assert conta["subtotal"] == Decimal("95.00")
```

```

assert conta["desconto"] == Decimal("9.50")
assert conta["total"] == Decimal("85.50")

def test_comanda_vazia_nao_quebra():
    assert fechar([])["total"] == Decimal("0.00")

def test_itens_validos_ignora_desconhecido():
    assert itens_validos(["corte", "pintura"]) == ["corte"]

```

NO TERMINAL
python -m pytest -q

SAÍDA ESPERADA

```

.....
5 passed in 0.01s

```

[100%]

ATENÇÃO

Os valores acima são da barbearia. Troque pelos números do domínio da SUA equipe. Um teste que confere o exemplo do professor, e não o seu sistema, não serve para nada.

POR QUE ISSO IMPORTA

O terceiro teste usa o valor EXATO da fronteira: três itens, que é o mínimo da regra. Defeito de software se concentra nas bordas, então esse é o teste que mais costuma pegar erro.

No VS Code, o ícone de frasco na barra lateral mostra a lista dos cinco testes e permite rodar um de cada vez.

CONFIRA ANTES DE SEGUIR

Cinco testes, todos verdes, com os números do domínio da sua equipe.

10 Fechar no Git 15 min

NO TERMINAL
cd ..
git status

ATENÇÃO

Antes de adicionar, confira o que apareceu. Devem estar `aula04/comanda.py` e `test_comanda.py`. Se aparecerem `.venv`, `__pycache__` ou `.pytest_cache`, o `.gitignore` da equipe precisa ser corrigido antes de seguir.

NO TERMINAL
git add aula04/comanda.py
git add aula04/test_comanda.py

git status

git commit -m "aula04: extrai funções da comanda e adiciona testes"
git push origin main

SAÍDA ESPERADA

```
[main 9c4b2f1] aula04: extrai funcoes da comanda e adiciona testes  
2 files changed, 61 insertions(+)
```

POR QUE ISSO IMPORTA

Se o push for recusado com fetch first, um colega enviou antes de voce. Rode git pull --rebase origin main e depois o push de novo.

CONFIRA ANTES DE SEGUIR

git status responde nothing to commit, working tree clean, e a pasta aula04 aparece no GitHub.

Se der errado

Os problemas abaixo respondem pela maior parte das travadas neste roteiro.

Sintoma	O que fazer
<code>ModuleNotFoundError: No module named comanda</code>	O pytest está sendo executado de fora da pasta aula04. Entre nela com <code>cd</code> antes de rodar. O import procura o arquivo a partir da pasta atual.
<code>pytest: command not found</code>	O ambiente virtual não está ativo, ou o pytest não foi instalado nele. Ative com <code>source .venv/bin/activate</code> e rode <code>pip install pytest</code> de novo. Prefira sempre a forma <code>python -m pytest</code> , que usa o Python do ambiente ativo.
O VS Code não encontra os testes	Selecione o interpretador correto: Cmd ou Ctrl mais Shift mais P, depois Python: Select Interpreter, e escolha o que está dentro de <code>.venv</code> . Em seguida, Python: Configure Tests, pytest, e a pasta atual.
<code>IndentationError: unexpected indent</code>	Mistura de espaços com tabulação. No VS Code, use Cmd ou Ctrl mais Shift mais P, Convert Indentation to Spaces. A convenção em Python é quatro espaços por nível.
O teste passa mesmo com o código errado	O teste provavelmente não está exercitando o caminho que você pensa. Quebre o código de propósito: se o teste continuar verde, ele não serve. Foi para isso que existiu o passo 8.
<code>TypeError</code> ao comparar <code>Decimal</code> com <code>float</code>	Não misture <code>Decimal("35.00")</code> com <code>35.00</code> . Todo valor monetário do arquivo precisa ser <code>Decimal</code> , inclusive o do teste.
O programa trava e não devolve o terminal	Provavelmente um <code>while</code> sem linha que aproxime o fim. Interrompa com Ctrl mais C e confira se algo dentro do laço realmente muda a condição.

Desafios opcionais

Para quem terminou antes do fim do laboratório. Não valem nota, mas costumam aparecer na prova em alguma forma.

- Acrescente uma função `aplicar_cupom(comanda, valor, codigo)` que zere o frete ou dê um desconto extra, e escreva o teste dela antes de escrever a função. Repare como o teste primeiro obriga a definir o contrato.
- Faça um teste que confirme que uma comanda com item repetido conta o item duas vezes no subtotal, mas as duas ocorrências como itens distintos apenas uma vez.
- Descubra o que acontece se você passar um texto em vez de uma lista para fechar. Escreva um teste que registre esse comportamento, seja ele qual for.
- Separe as funções em dois arquivos, um com o catálogo e outro com o cálculo, e ajuste os imports. Rode a suíte de novo: ela precisa continuar verde sem nenhuma alteração nos testes.

Entrega

O que entregar	A pasta aula04 no repositório da equipe, com <code>comanda.py</code> e <code>test_comanda.py</code> , adaptados ao domínio escolhido pela equipe na Aula 1.
Registro obrigatório	No corpo do envio no ambiente virtual, cole as duas saídas do pytest do passo 8: a que falhou, com o <code>AssertionError</code> , e a que passou depois da correção.
Prazo	Domingo, às 23h59.
Formato do envio	Um envio por equipe, com o link do repositório. Todos os integrantes precisam ter pelo menos um commit da semana.

Critério de aceite

- As quatro funções existem e nenhuma delas tem print no corpo.
- O desconto olha para a quantidade de itens válidos, e não para o tamanho da comanda inteira.
- A suíte tem no mínimo cinco testes e todos passam.
- Existe um teste para a coleção vazia e um teste no valor exato da fronteira da regra.
- Os valores dos testes são do domínio da equipe, e não os do exemplo da barbearia.
- A saída do pytest com o teste falhando foi registrada no envio.
- Nenhum arquivo de ambiente virtual, cache ou compilado foi enviado ao repositório.

Para consultar durante o laboratório

Python, controle de fluxo: for, while, break e continue

<https://docs.python.org/pt-br/3/tutorial/controlflow.html>

Python, módulos e a variável `__name__`

<https://docs.python.org/pt-br/3/tutorial/modules.html>

Python, FAQ: por que valores padrão são compartilhados entre chamadas

<https://docs.python.org/3/faq/programming.html#why-are-default-values-shared-between-objects>

pytest, guia de primeiros passos

<https://docs.pytest.org/en/stable/getting-started.html>

Real Python, o que faz o `if __name__ == '__main__':`

<https://realpython.com/if-name-main-python/>

Martin Fowler, catálogo de refatoração: extrair função

<https://refactoring.com/catalog/extractFunction.html>

Visual Studio Code, testes em Python

<https://code.visualstudio.com/docs/python/testing>

RAMALHO, Luciano. Python Fluente. 2. ed. Novatec, 2023. Capítulo 7

<https://novatec.com.br/livros/python-fluente-2ed/>